

Министерство высшей науки и образования РФ

Политехнический институт

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3 по курсу «Логика и основы
алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнил: ст. гр. 25BBB2

Тургенев И.В.

Козлов А.В.

Проверил: к.т.н. доцент

Юрова О.В.

к.т.н. доцент

Деев М.В.

Пенза 2026

Название

Динамические списки

Цель работы:

Изучение основных типов динамических списков и методов их обработки

Лабораторное задание:

Задание 1: реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Задание 2: на основе приведенного кода реализуйте структуру данных Очередь.

Задание 3: на основе приведенного кода реализуйте структуру данных Стек.

Описание использованных методов для решения заданий

Задание 1:

`#include <stdio.h>` подключает функции `printf` и `scanf`. `#include <stdlib.h>` — функции `malloc` и `free`. `#include <string.h>` — `strcpy` и `strcmp`. `#include <windows.h>` — настройку кодировки консоли.

`struct node` описывает узел: поле `inf[256]` для названия, указатель `next` и целочисленное поле `priority`.

Глобальные указатели `head` и `last` хранят начало и конец списка.

Функция `get_struct()` выделяет память под новый узел через `malloc`, запрашивает название и приоритет, копирует строку в `p->inf` через `strcpy`.

Функция `spstore()` вставляет элемент с сохранением порядка по убыванию приоритета: если список пуст — узел становится головой и хвостом; если приоритет больше головы — вставляется перед ней; иначе список просматривается циклом `while`, пока не найдётся место, где приоритет следующего меньше нового.

Функция `review()` перебирает список циклом `while (struc)` и выводит имя и приоритет каждого элемента.

Функция `del(name)` удаляет элементы с заданным именем: указатель `prev` хранит предыдущий узел, при совпадении имени перенастраиваются указатели `head` или `prev->next`, при удалении хвоста обновляется `last`, память

освобождается через free.

Функция main() настраивает кодировку, трижды вызывает spstore(), выводит список, запрашивает имя и вызывает del(name).

Метод решения — односвязный список с упорядоченной вставкой по приоритету.

Задание 2:

Библиотеки те же. struct node содержит inf[256] и next.

Глобальные head и last — начало и конец очереди.

Функция get_struct() выделяет память и считывает значение элемента.

Функция enqueue() добавляет элемент в конец: если очередь пуста — head и last указывают на новый узел; иначе новый узел присоединяется после last, и last переводится на него.

Функция dequeue() извлекает элемент из начала: сохраняет указатель на голову, выводит её значение, переводит head на следующий элемент, при опустошении обнуляет last, освобождает память через free.

Функция review() перебирает очередь циклом while (struc != NULL) и выводит значения от начала к концу.

Функция clear() освобождает память всех узлов и обнуляет head, last.

Метод решения — односвязный список с двумя указателями, обеспечивающий дисциплину FIFO.

Задание 3:

Библиотеки те же. struct node содержит inf[256] и next.

Глобальный head — вершина стека.

Функция get_struct() выделяет память и считывает значение элемента.

Функция push() добавляет элемент на вершину: новый узел получает указатель на бывшую вершину (p->next = head), затем head переводится на новый узел.

Функция pop() извлекает элемент с вершины: сохраняет указатель, выводит значение, переводит head на следующий, освобождает память через free.

Функция review() перебирает стек от вершины к основанию циклом while (struc != NULL).

Функция clear() освобождает память всех узлов и обнуляет head.

Метод решения — односвязный список с одним указателем на вершину, обеспечивающий дисциплину LIFO.

Результаты работы программы

Задание 1:

```
Консоль отладки Microsoft Visual Studio
Введите название объекта:
A
Введите приоритет объекта:
1
Введите название объекта:
B
Введите приоритет объекта:
2
Введите название объекта:
C
Введите приоритет объекта:
3
Имя - C, приоритет - 3
Имя - B, приоритет - 2
Имя - A, приоритет - 1
Введите имя для удаления: B
Имя - C, приоритет - 3
Имя - A, приоритет - 1

C:\Users\User\Desktop\Логика и основы алгоритмизации в инженерных задачах\Logic-and-fundamentals-of-algorithmization-in-engineering-tasks\tasks\task1\x64\Debug\task1.exe (процесс 8896) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно...
```

Задание 2:

```
Консоль отладки Microsoft Visual Studio
Введите значение элемента:
A
Введите значение элемента:
B
Введите значение элемента:
C
Очередь (начало -> конец):
A
B
C
Извлечён элемент: A
Извлечён элемент: B
Очередь (начало -> конец):
C

C:\Users\User\Desktop\Логика и основы алгоритмизации в инженерных задачах\Logic-and-fundamentals-of-algorithmization-in-engineering-tasks\tasks\task2\x64\Debug\task2.exe (процесс 18396) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно...
```

Задание 3:

```
Консоль отладки Microsoft Visual Studio
Введите значение элемента:
A
Введите значение элемента:
B
Введите значение элемента:
C
Стек (вершина -> основание):
C
B
A
Извлечён элемент: C
Извлечён элемент: B
Стек (вершина -> основание):
A

C:\Users\User\Desktop\Логика и основы алгоритмизации в инженерных задачах\Logic-and-fundamentals-of-algorithmization-in-engineering-tasks\tasks\task3\x64\Debug\task3.exe (процесс 808) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно...
```

Выводы

В ходе выполнения лабораторной работы были изучены и реализованы основные структуры данных на основе односвязного списка: приоритетный список, очередь (FIFO) и стек (LIFO). Освоены динамическое выделение памяти, работа с указателями и операции вставки, извлечения, просмотра и удаления элементов.